

UNIVERSIDADE ESTADUAL DE MATO GROSSO DO SUL
CÂMPUS DOURADOS
CIÊNCIA DA COMPUTAÇÃO



LUÍS EDUARDO SOUZA VASCONCELOS

RELATÓRIO TRABALHO 1 DE PROGRAMAÇÃO PARALELA E DISTRIBUÍDA
(Encontrar a árvore geradora mínima utilizando o MPI)

DOURADOS - MS
2023

1. Sumário do problema

Este trabalho tem como objetivo abordar a resolução paralela do problema de encontrar a árvore geradora mínima, utilizando o MPI. A tarefa consiste em identificar, a partir de um arquivo de entrada que contém informações sobre o número de vértices, arestas e o conjunto de arestas, a árvore geradora mínima.

Ao longo do desenvolvimento deste trabalho, nos deparamos com desafios significativos. Um deles foi compreender os tipos abstratos de dados e interpretar os algoritmos fornecidos como base. Além disso, enfrentamos o desafio de determinar quais partes do código poderiam ser paralelizadas e como realizar essa paralelização, levando em consideração a comunicação entre as máquinas.

Para garantir o correto funcionamento da implementação, uma estrutura de dados fundamental foi o union-find, que permitiu uma implementação enxuta do algoritmo de busca da árvore geradora mínima em si.

2. Uma descrição sucinta dos algoritmos e dos tipos abstratos de dados, das principais funções, procedimentos e das decisões de implementação

As duas principais estruturas de dados utilizadas foram:

```
typedef struct node {
    struct node *raiz;
    int rank;
} Conjunto; // estrutura utilizada para a utilização do
union-find

typedef struct {
    u_int64_t u;
    u_int64_t v;
    u_int64_t peso;
} Aresta; // estrutura para a representação das arestas
```

A primeira representa os conjuntos (componentes) para a formação da árvore geradora mínima. Essa estrutura é utilizada especificamente para o funcionamento da estrutura union-find.

Já a segunda estrutura é utilizada para a representação das arestas de grafos, sendo a única necessária para o funcionamento do algoritmo nesta versão

do código. Posteriormente no tópico 3, irei apresentar outras versões e decisões de implementação para esse trabalho.

Também foi utilizado o recurso do MPI para a representação das arestas:

```
MPI_Datatype MPI_Aresta; // tipo de aresta do MPI
MPI_Datatype MPI_Vertice; // tipo de vertice do MPI
```

O **MPI_Aresta** é utilizado para facilitar o envio e também recebimento das arestas durante a comunicação entre os processos, é possível utilizar esse tipo de dados ao invés de utilizar o tipo `MPI_BYTE`, que é o tipo de dados mais genérico.

A inicialização desses tipos foi feita na função de inicialização:

```
void inicializacao(int argc, char **argv) {
    // Inicializando MPI
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    // Criando o tipo de aresta MPI
    MPI_Type_contiguous(3, MPI_UINT64_T, &MPI_Aresta);
    MPI_Type_commit(&MPI_Aresta);

    // Criando o tipo de vertice MPI
    MPI_Type_contiguous(2, MPI_UINT64_T, &MPI_Vertice);
    MPI_Type_commit(&MPI_Vertice);
}
```

Uma das partes principais para a execução do algoritmo em paralelo, é a distribuição das arestas entre os processos, ou seja, a distribuição de tarefas. Para isso, cada processo precisa de uma parcela do número total de arestas para encontrar a árvore geradora mínima (AGM).

Após identificar que os vértices do arquivo são organizados de forma sequencial, isto é, de 0 à 1965205, foi possível então definir para cada processo um intervalo de vértices no qual ele seria responsável por processar. Dessa forma eu consegui eliminar um dos principais problemas que seria a distribuição das arestas entre os processos, visto que se as arestas fossem distribuídas com parcelas regulares, teríamos o problema em que um processo não tivesse todas as arestas necessárias ligadas a algum vértice, comprometendo assim o cálculo de AGM.

A distribuição das arestas é feita pelo procedimento:

```
void distribuirArestasPorProcessador() {
    // encontrando o intervalo de vertice de cada rank
    u_int64_t verticePorProcessador = totalVerticesGlobal / size;
    u_int64_t primeiroVertice = rank * verticePorProcessador;
    u_int64_t ultimoVertice = (rank + 1) * verticePorProcessador;

    // Último processador fica com o restante dos vértices (se houver)
    if (rank == size - 1) {
        ultimoVertice += totalVerticesGlobal % size;
    }
}
```

A seguir é mostrado o laço em que é feita a leitura do arquivo de arestas e adicionado as respectivas arestas que pertencem ao intervalo de vértices do processo. O intervalo é verificado com a macro definida:

```
#define RANGE(vertice, inferior, superior) ((vertice) >= (inferior) && (vertice) < (superior))
```

```
Aresta tmp;
for (u_int64_t i = 0; i < totalArestasGlobal; i++) {
    fscanf(stream: arquivo, format: "%lu %lu %lu", &tmp.v, &tmp.u, &tmp.peso);
    arestasCompleto[i] = tmp;
    // verifica se o vertice está dentro do intervalo calculado para cada rank (desse modo garante que cada rank tem t
    if (RANGE(tmp.v, primeiroVertice, ultimoVertice) || RANGE(tmp.u, primeiroVertice, ultimoVertice)) {
        // realoca a quantidade de arestas
        if (quantidadeArestasLocal == quantidadeArestasInicial) {
            quantidadeArestasInicial *= 2;

            arestasLocal = (Aresta *) realloc(ptr: arestasLocal, size: quantidadeArestasInicial * sizeof(Aresta));

            if (arestasLocal == NULL) {
                abortProgram(mensagem: "\n[ERRO] erro ao relocal arestas\n");
            }
        }
        arestasLocal[quantidadeArestasLocal++] = tmp;
    }
}

fclose(stream: arquivo);
// garantir que todos os ranks já leram o arquivo
MPI_Barrier(comm: MPI_COMM_WORLD);
```

Após sair do procedimento de leitura do arquivo, entramos no laço principal, onde de fato é realizado o cálculo da árvore geradora mínima.

```
encontrarAGMLocal();

while (processadores > 1) {

    // determina qual grupo os ranks pertencem (remetente destinatário)
    if ((rank / (int) pow(2, y: it)) % 2 != 0) {
        destination = (rank - (int) pow(2, y: it));

        MPI_Send(buf: arvoreGeradoraMinimaLocal, count: quantidadeArestasAGMLocal, datatype: MPI_Aresta, dest: destination, tag: 0, comm: MPI_COMM_WORLD);
        // debug("[ITERAÇÃO: %d] Enviando a arvore para o rank %d -- SAINDO...\n", it, destination);
        break;
    } else {
        source = (rank + (int) pow(2, y: it)); // rank of source

        // Recebe as arestas na segunda metade do espaço alocado
        MPI_Recv(buf: mergedAGM + quantArestasMergedAGM, count: totalVerticesGlobal - 1, datatype: MPI_Aresta, source, tag: 0, comm: MPI_COMM_WORLD, &status);
        MPI_Get_count(&status, datatype: MPI_Aresta, count: &quantArestasRecvAGM);
        // debug("[ITERAÇÃO: %d] Recebendo agm do rank: %d\n", it, source);
    }
    quantArestasMergedAGM += quantArestasRecvAGM;

    encontrarAGMParalelo();

    processadores /= 2; // a cada iteração a metade dos processadores saem dos calculos (são os que enviam a agm para outro rank). Isso pq metade recebe e it++;
}
```

Antes de entrar no laço cada processo calcula a sua árvore geradora com a suas arestas locais, que foram obtidas durante a leitura do arquivo:

```
encontrarAGMLocal();
```

Após isso, os processos entram no laço e realizam o calcula da AGM em paralelo na função:

```
encontrarAGMParalelo();
```

Essa função realiza o mesmo procedimento que na função `encontrarAGMLocal()`; a única diferença entre as duas, é que na em paralelo o processo vai usar tanto a árvore local dele, quanto a árvore recebida de outro processo para calcular a AGM.

Para conseguir fazer tudo isso funcionar, foi necessário muita depuração, então foi criado uma função especificamente para isso:

```
// função auxiliar para depuração
void debug(char *format, ...) {
    va_list args;

    va_start(args, format);

    printf(format: "%6.6f|%2d|", get_timer(), rank);
    vprintf(format, arg: args);

    va_end(args);
}
```

Para isso foi utilizado a biblioteca `stdarg.h` e também a `time.h`. Essa função fazer o print do rank, o tempo de execução e também qualquer outra informação que passar no format.

3.Decisões de implementação

Como mencionado anteriormente, esse trabalho teve dois fluxos de implementação, a descrição acima foi do segundo fluxo, que foi possível por causa da verificação em que os vértices do arquivo são organizados de forma sequencial. Antes de percebido esse padrão, o trabalho foi implementado de uma forma diferente e também ineficiente, demorando até 40 minutos para conseguir distribuir as arestas entre os processos.

Para a distribuição das arestas foi feita em duas etapas:

1) Distribuir as arestas entre os processos

A distribuição era responsabilidade do rank 0 (ROOT), ele fazia a leitura de todo o arquivo e depois distribuía entre os ranks utilizando a função coletiva *Scatterv*. Após todos os ranks receberem sua parcela de arestas, eles percorriam essa parcela guardando todos os vértices da parcela.

2) Encontrar arestas faltantes

Apesar da distribuição ser feito de forma relativamente rápida, ainda teria o problema a ser tratado para o caso dos vértices que não possuíssem todas as suas ligações necessárias. Para isso foi criado o procedimento:

```
void encontrarArestasFaltantes();
```

Ele seria o responsável por tratar esse problema. Porém a sua implementação não foi nada fácil. Primeiro foi necessário que cada processo tivesse acesso tanto a quantidade de arestas quanto as arestas em si.

```
// Coletar a quantidade de arestas e arestasLocal de cada
processo no processo raiz
MPI_Gather(&quantidadeArestasLocal, 1, MPI_UINT64_T,
          quantidadeArestasRanks, 1, MPI_UINT64_T,
          ROOT, MPI_COMM_WORLD);

// Transmitir a quantidade de vértices e arestasLocal de
cada processo para todos os outros processos
MPI_Bcast(quantidadeArestasRanks, size, MPI_UINT64_T,
          ROOT, MPI_COMM_WORLD);
```

Após todos os processos terem recebido a quantidade de arestas que cada processo tem, então ficaria mais fácil de obter as arestas de cada processo utilizando a função coletiva MPI `MPI_Allgatherv()`; Após ter recebido as arestas de cada, também seria necessário que os ranks tivesse, acesso aos vértices totais do arquivo, para assim conseguir identificar se o grau do vértice a ser analisado está completo. O algoritmo funciona da seguinte forma:

```
para cada arestaLocal faca:
    se verticeLocal.grau == verticeGlobal.grau entao:
        continue
    fimse

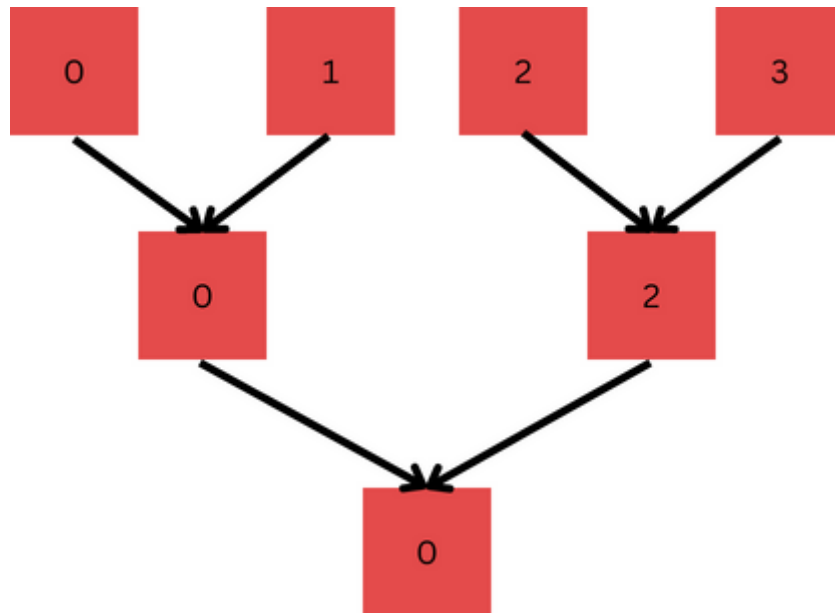
    para cada arestaDosProcessos diferente das arestaLocal faca:
        se verticeLocal.grau == verticeGlobal.grau entao:
            break;
        fimse

        se verticeLocal == aresta.u ou verticeLocal == aresta.v entao
            adicione aresta no conjunto local
            incremente em 1 o grau do verticeLocal
        fimse
    fimpara
fimpara
```

No final do algoritmo eram obtidas todas as arestas faltantes, porém o algoritmo tem-se uma complexidade de $O(n^2)$ sendo n o número de arestas. Uma possível melhoria seria não percorrer as arestas dos processos que não possuem o verticeLocal no seu conjunto de vértices, todavia não iria diminuir significativamente o tempo de execução.

4. Comunicação entre as máquinas

A comunicação entre as máquinas foi feita de uma maneira simples, seguinte a seguinte estrutura:



Para isso funcionar foi utilizada uma fórmula para determinar em cada iteração em qual grupo os processos pertencem, enviar/receber.

```
// determina qual grupo os ranks pertencem (remetente destinatário)
if ((rank / (int) pow(2, y: it)) % 2 != 0) {
    destination = (rank - (int) pow(2, y: it));

    MPI_Send(buf: arvoreGeradoraMinimaLocal, count: quantidadeArestasAGMLocal, datatype: MPI_Aresta, des
    // debug("[ITERAÇÃO: %d] Enviando a arvore para o rank %d -- SAINDO...\n", it, destination);
    break;
} else {
    source = (rank + (int) pow(2, y: it)); // rank of source

    // Recebe as arestas na segunda metade do espaço alocado
    MPI_Recv(buf: mergedAGM + quantArestasMergedAGM, count: totalVerticesGlobal - 1, datatype: MPI_Aresta
    MPI_Get_count(&status, datatype: MPI_Aresta, count: &quantArestasRecvAGM);
    // debug("[ITERAÇÃO: %d] Recebendo agm do rank: %d\n", it, source);
}
quantArestasMergedAGM += quantArestasRecvAGM;
```

Essa foi a fórmula utilizada:

```
((rank / (int) pow(2, it)) % 2 != 0)
```

Nela, a cada iteração um novo grupo é definido, o interessante aqui é que sempre o rank 0 (ROOT) que no final irá ter o resultado da AGM.

Após uma máquina enviar a AGM local dela, ela pode sair do loop, parando de processar, deixando o restante do processamento para o grupo de processadores que receberam a AGM parcial.

Uma limitação dessa abordagem é que só é possível fazer a execução com o número de processadores sendo uma potência de 2.

5.Características da máquina utilizada nos testes

O testes do programa foram executados em um computador com as seguintes características:

Processador: Intel Core i7-12700 KF (12 núcleos e 20 threads).

Clock do processador: 5.0GHz

Memória: 16 Gb.

Frequência da memória: 3200MHz

Armazenamento: SSD 256 GB.

Tempo de leitura/escrita: 3000 MB/s

Chipset: Z690

6.Testes e execução do programa

Para realizar o testes de execução e obter os tempos de execução, foi utilizado o comando nativo do linux “time” que ao colocar ele antes da execução do programa, ele retorna o tempo de execução do programa, por exemplo:

```
time ./main dados_entrada_sequencial.txt
```

O valor retornado vem com a seguinte formatação:

```
real 0m0,003s
```

```
user 0m0,003s
```

```
sys 0m0,000s
```

Por fins de simplicidade, só será apresentado o valor “**real**”.

6.1 Algoritmo sequencial com dados de entrada sequencial

Nesses dados tem-se um total de 91 vértices e 98 arestas.

Usando a média de 5 execuções utilizar, obteve-se o valor de:

```
real 0m0,003s
```

Neste caso, trata-se de um algoritmo sequencial com um conjunto de dados de entrada também sequencial. Não há paralelismo envolvido nessa execução, portanto não é possível calcular o speedup ou a eficiência.

6.2 Algoritmo sequencial com dados de entrada paralelo

Nesses dados tem-se um total de 1965206 vértices e 2769244 arestas.

Não foi possível fazer o teste do algoritmo sequencial com os dados de entrada paralelo, pois a execução demora muito. Isso provavelmente se deve ao fato de que o tempo de leitura do algoritmo sequencial implementado é $O(n^2)$.

6.3 Algoritmo paralelo com dados de entrada sequencial

Serão realizados os testes utilizando as threads do sistema, pois o processador possui apenas 12 núcleos, sendo assim não será possível realizar os teste para 16 máquinas. Para ser possível utilizar as threads do sistema e não os núcleos que vem por padrão, é necessário adicionar uma flag na hora da execução do código: **--use-hwthread-cpus**. Ficando assim um exemplo de comando:

```
time mpirun --use-hwthread-cpus -np 1 ./main arquivo2.txt
```

6.3.1 Utilizando 1 thread (média de 5 execuções)

real 0m0,286s

6.3.2 Utilizando 2 threads (média de 5 execuções)

real 0m0,287s

6.3.3 Utilizando 4 threads (média de 5 execuções)

real 0m0,288s

6.3.4 Utilizando 8 threads (média de 5 execuções)

real 0m0,293s

6.3.5 Utilizando 16 threads (média de 5 execuções)

real 0m0,298s

Speedup = Tempo Sequencial / Tempo Paralelo.

Eficiência = Speedup / Número de Threads.

Considerando o tempo médio de execução com 1 thread (0.286s) como referência, podemos calcular o speedup e a eficiência para cada caso:

6.3.1.1 Utilizando 1 thread:

- Speedup = $0.286s / 0.286s = 1$
- Eficiência = $1 / 1 = 1$ (100%)

6.3.2.1 Utilizando 2 threads:

- Speedup = $0.286s / 0.287s \approx 0.996$

- Eficiência = $0.996 / 2 \approx 0.498$ (49.8%)

6.3.3.1 Utilizando 4 threads:

- Speedup = $0.286s / 0.288s \approx 0.993$
- Eficiência = $0.993 / 4 \approx 0.248$ (24.8%)

6.3.4.1 Utilizando 8 threads:

- Speedup = $0.286s / 0.293s \approx 0.976$
- Eficiência = $0.976 / 8 \approx 0.122$ (12.2%)

6.3.5.1 Utilizando 12 threads:

- Speedup = $0.286s / 0.298s \approx 0.959$
- Eficiência = $0.959 / 12 \approx 0.08$ (8%)

6.4 Algoritmo paralelo com dados de entrada paralelo

- Total de vértices: 1,965,206
- Total de arestas: 2,769,244

6.4.1 Utilizando 1 thread (média de 5 execuções)

real 0m1,343s

6.4.2 Utilizando 2 threads (média de 5 execuções)

real 0m1,885s

6.4.3 Utilizando 4 threads (média de 5 execuções)

real 0m1,394s

6.4.4 Utilizando 8 threads (média de 5 execuções)

real 0m1,338s

6.4.5 Utilizando 16 threads (média de 5 execuções)

real 0m1,832s

Considerando o tempo médio de execução com 1 thread (1.343s) como referência, podemos calcular o speedup e a eficiência para cada caso:

6.4.1.1 Utilizando 1 thread:

- Speedup = $1.343s / 1.343s = 1$
- Eficiência = $1 / 1 = 1$ (100%)

6.4.2.1 Utilizando 2 threads:

- Speedup = $1.343s / 1.885s \approx 0.711$
- Eficiência = $0.711 / 2 \approx 0.355$ (35.5%)

6.4.3.1 Utilizando 4 threads:

- Speedup = $1.343s / 1.394s \approx 0.962$

- Eficiência = $0.962 / 4 \approx 0.241$ (24.1%)

6.4.4.1 Utilizando 8 threads:

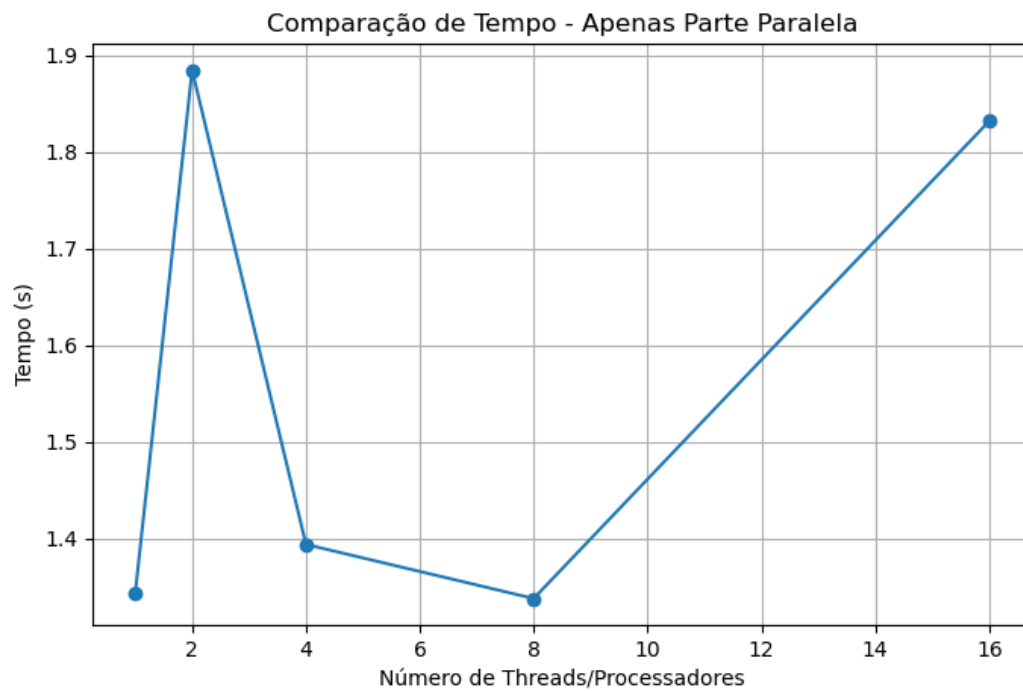
- Speedup = $1.343s / 1.338s \approx 1.004$
- Eficiência = $1.004 / 8 \approx 0.125$ (12.5%)

6.4.5.1 Utilizando 12 threads:

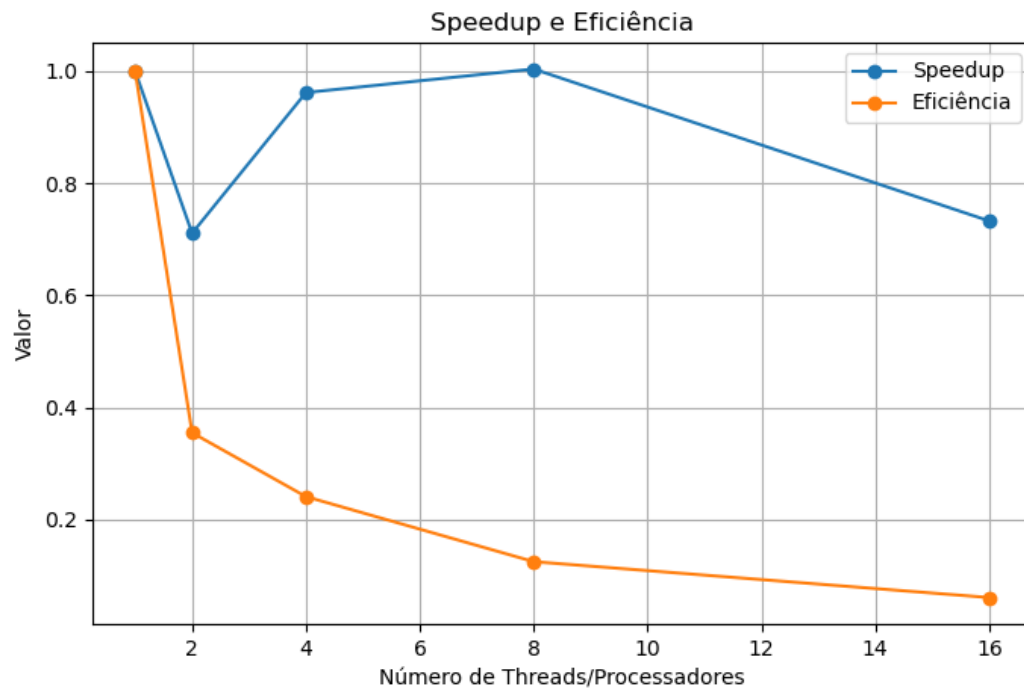
- Speedup = $1.343s / 1.832s \approx 0.733$
- Eficiência = $0.733 / 12 \approx 0.061$ (6.1%)

6.5 Gráficos

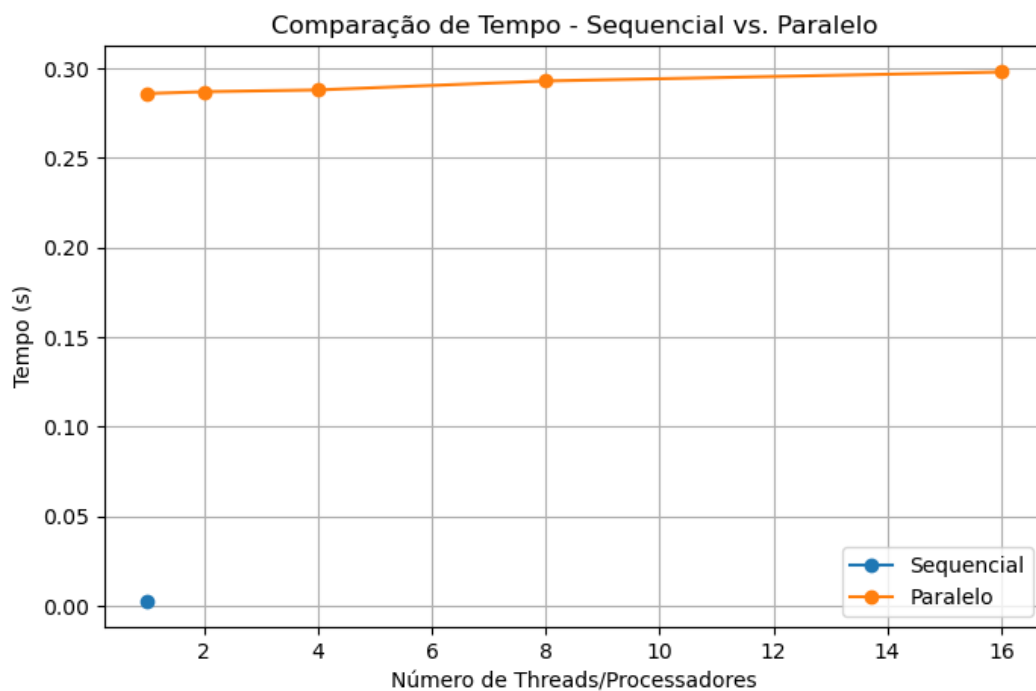
Comparação do tempo/threads utilizando apenas com o arquivo de dados paralelo.



Speedup e Eficiência do processamento apenas com o arquivo de dados paralelo.



Comparação do tempo de execução do código sequencial utilizando o arquivo de dados sequencial e do código em paralelo utilizando o arquivo de dados sequencial.



7. Imagens do funcionamento do programa

7.1 Execução do algoritmo sequencial

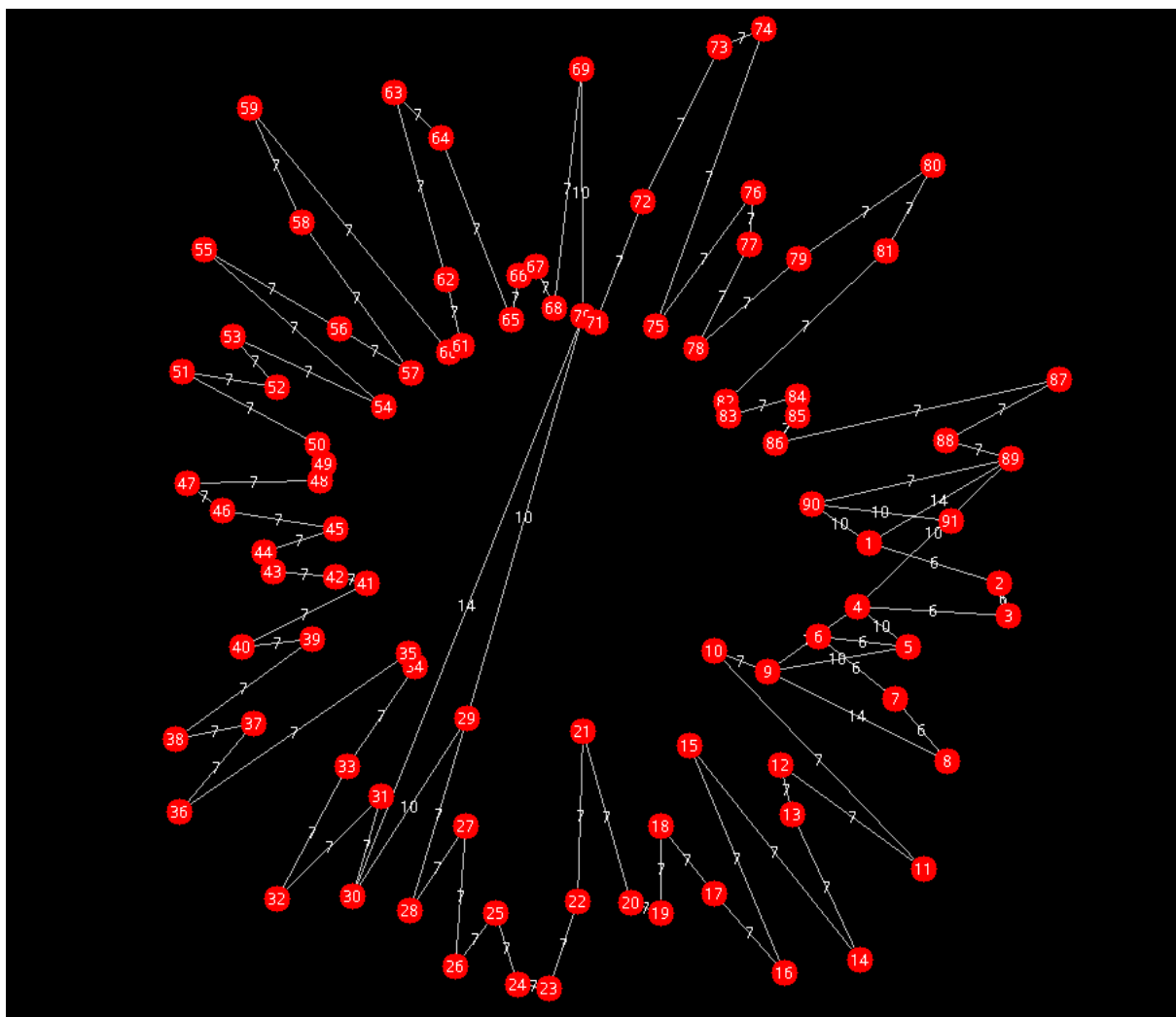
Compilando:

```
luise@luise-desktop:~/CLionProjects/ppd-2023-seq$ make clean
Removing object files
Removing applications
Removing ofiles directory
luise@luise-desktop:~/CLionProjects/ppd-2023-seq$ make all
Building ofiles/grafo.o...
Building ofiles/sdl_utils.o...
Building main...
luise@luise-desktop:~/CLionProjects/ppd-2023-seq$
```

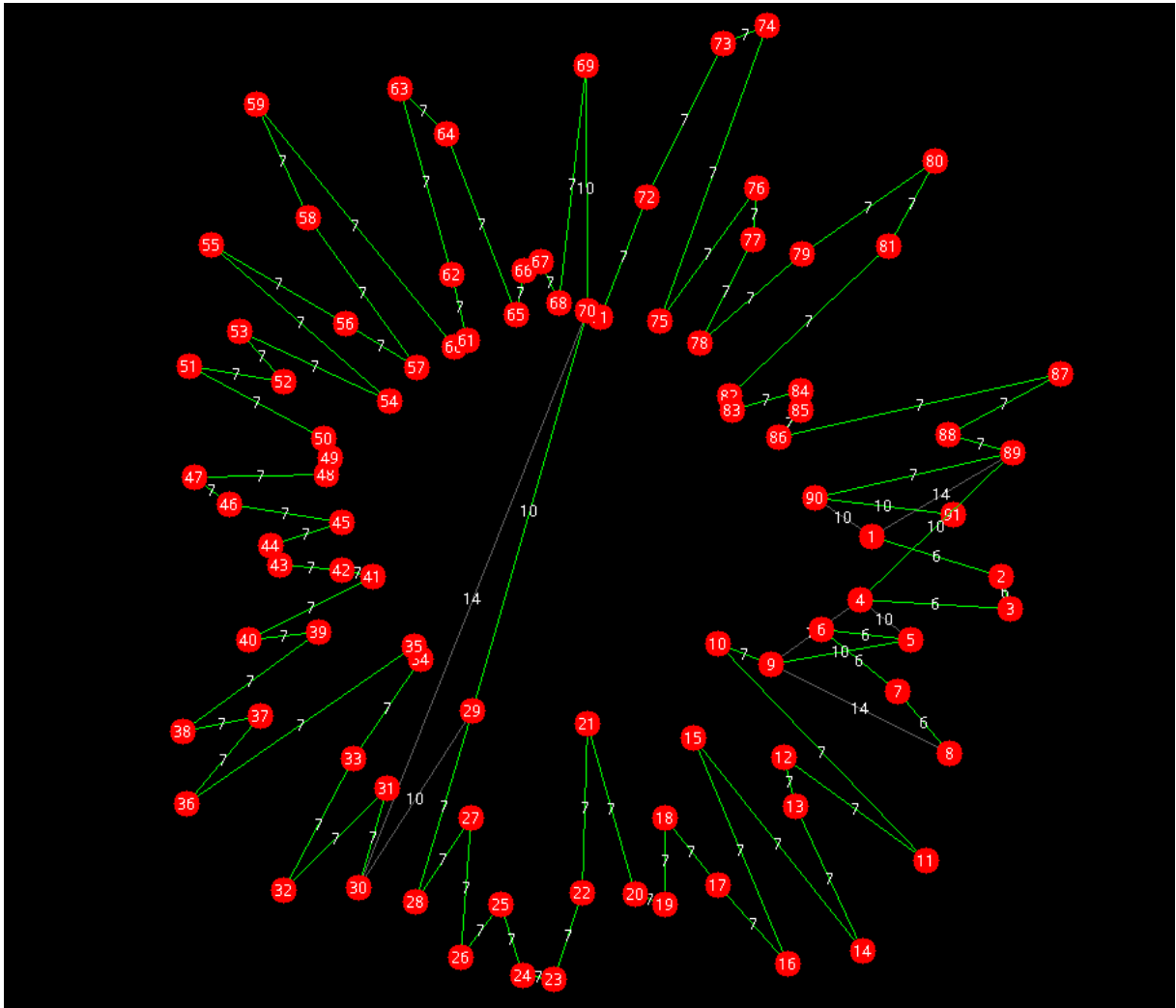
Executando:

```
luise@luise-desktop:~/CLionProjects/ppd-2023-seq$ time ./main dados_entrada_sequencial.txt
Comandos:
- Espaço: Mostrar AGM
- Seta para cima: Aumentar seed
- Seta para baixo: Diminuir seed
- R: Resetar seed
Grafo AGM:
Número de vértices: 91
Número de arestas: 90
Arestas:
Peso total: 642
```

Grafo completo:



Árvore geradora mínima destacado em verde:



7.2 Execução do algoritmo paralelo

Compilando:

```
luise@luise-desktop:~/CLionProjects/ppd_2023_par$ mpicc -o main main.c -lm -Wextra -Wall -pedantic
main.c: In function 'distribuirArestasPorProcessador':
main.c:176:82: warning: comparison of integer expressions of different signedness: 'int' and 'u_int64_t'
176 |     u_int64_t quantidadeArestasInicial = quantidadeArestasPorProcessador + (rank < restanteAres
    |                                                                    ^
luise@luise-desktop:~/CLionProjects/ppd_2023_par$
```

Executando para 8 núcleos:

```
luise@luise-desktop:~/CLionProjects/ppd_2023_par$ time mpirun -np 8 ./main super_arquivo.txt

1.047188| 0|Quantida de arestas da AGM: 1965205
1.130966| 0|Peso parcial/total: 3222179092953490

real    0m1,426s
user    0m6,752s
sys     0m0,361s
luise@luise-desktop:~/CLionProjects/ppd_2023_par$
```

8. Conclusão

O presente trabalho teve como objetivo desenvolver uma implementação paralela utilizando o MPI para encontrar árvores geradoras mínimas. Durante o desenvolvimento, foram enfrentadas diversas dificuldades, principalmente na compreensão dos tipos abstratos de dados e na interpretação dos algoritmos fornecidos como base.

Uma das principais decisões de implementação foi a escolha da estrutura de dados Union-Find, que se mostrou fundamental para facilitar a implementação e tornar o algoritmo de busca da árvore geradora mínima mais eficiente. Essa estrutura de dados permitiu reduzir o código do algoritmo, tornando-o mais conciso e de fácil compreensão. Outra importante decisão foi ter alterado a forma de distribuição das arestas entre os processadores, utilizando o intervalo de vértices, tornando a distribuição de dados muito simples.

Para avaliar o desempenho da implementação paralela, foram realizados testes com diferentes configurações de threads/processadores. Os resultados obtidos demonstraram que o algoritmo paralelo apresentou speedup em relação ao algoritmo sequencial. No entanto, o speedup não foi linear em todos os casos, indicando possíveis limitações na escalabilidade do algoritmo para um grande número de threads/processadores. Além disso, a eficiência também diminuiu à medida que mais threads/processadores foram utilizados.

Dessa forma, conclui-se que o trabalho atingiu seu objetivo de implementar uma solução paralela para o problema de encontrar árvores geradoras mínimas. A estrutura de dados Union-Find e a forma de distribuição das arestas foram cruciais para simplificar a implementação e tornar o algoritmo mais eficiente. No entanto, são

necessárias análises mais aprofundadas e otimizações adicionais para lidar com as limitações de escalabilidade e melhorar a eficiência do algoritmo em cenários com um grande número de threads/processadores.

O trabalho contribui para a compreensão dos desafios e benefícios da implementação paralela de algoritmos complexos, fornecendo insights valiosos sobre a importância da escolha adequada das estruturas de dados e da consideração dos aspectos de comunicação em ambientes paralelos.

Referências bibliográficas

<https://www.open-mpi.org/doc/v4.1/>

<https://www.youtube.com/watch?v=ayW5B2W9hfo>